

```
In [1]: # # Install spacy
# !pip install spacy
# !spacy download en_core_web_Lg
```

```
In [2]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [3]: df = pd.read_json("News_Category_Dataset_v3.json", lines=True)
df.head()
```

```
Out[3]:
```

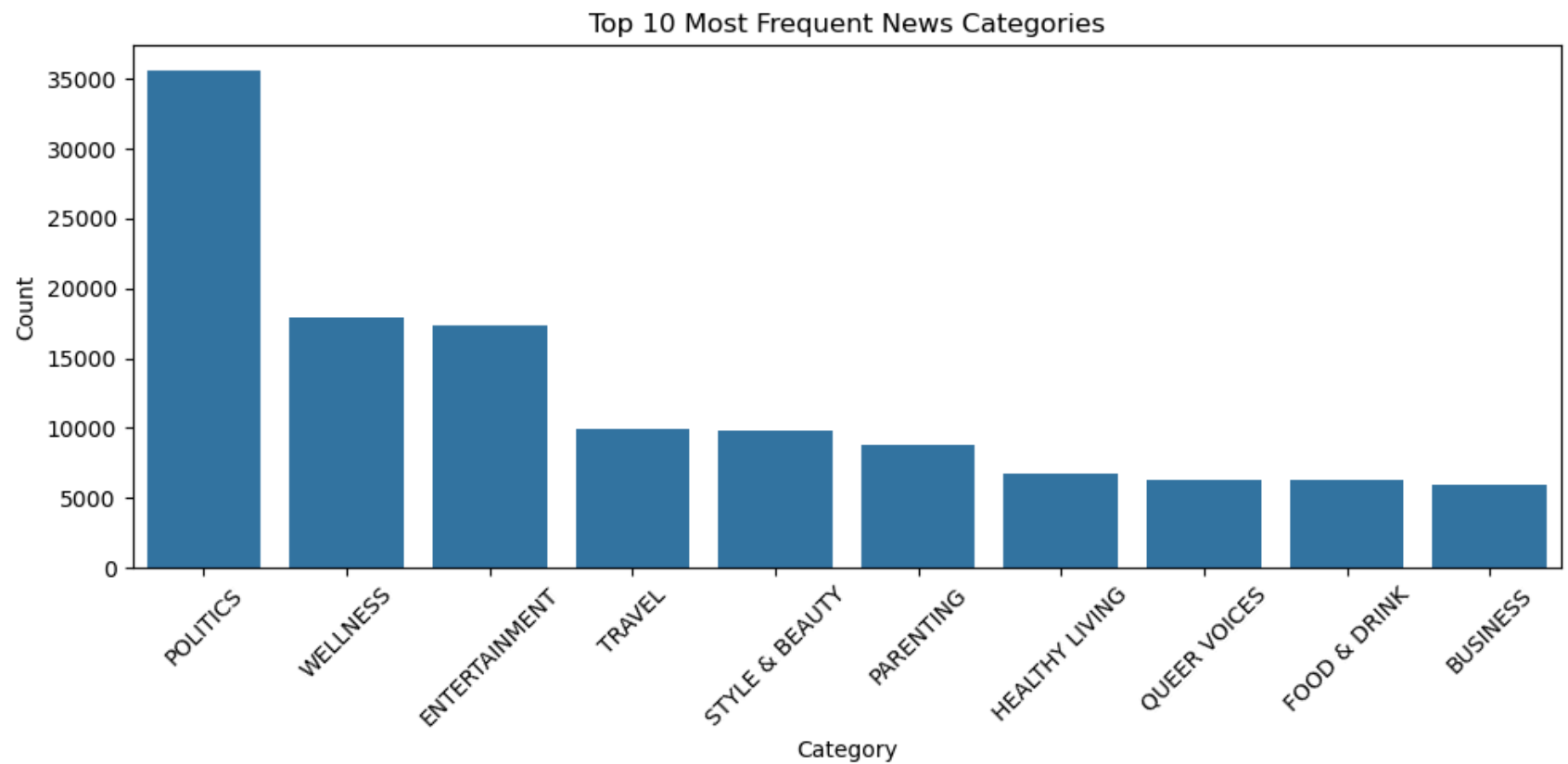
	link	headline	category	short_description	authors	date
0	https://www.huffpost.com/entry/covid-boosters-...	Over 4 Million Americans Roll Up Sleeves For O...	U.S. NEWS	Health experts said it is too early to predict...	Carla K. Johnson, AP	2022-09-23
1	https://www.huffpost.com/entry/american-airlin...	American Airlines Flyer Charged, Banned For Li...	U.S. NEWS	He was subdued by passengers and crew when he ...	Mary Papenfuss	2022-09-23
2	https://www.huffpost.com/entry/funniest-tweets...	23 Of The Funniest Tweets About Cats And Dogs ...	COMEDY	"Until you have a dog you don't understand wha...	Elyse Wanshel	2022-09-23
3	https://www.huffpost.com/entry/funniest-parent...	The Funniest Tweets From Parents This Week (Se...	PARENTING	"Accidentally put grown-up toothpaste on my to...	Caroline Bologna	2022-09-23
4	https://www.huffpost.com/entry/amy-cooper-lose...	Woman Who Called Cops On Black Bird-Watcher Lo...	U.S. NEWS	Amy Cooper accused investment firm Franklin Te...	Nina Golgowski	2022-09-22

```
In [62]: import seaborn as sns
import matplotlib.pyplot as plt

top10 = df['category'].value_counts().head(10).index

plt.figure(figsize=(10, 5))
```

```
sns.countplot(  
    data=df,  
    x='category',  
    order=top10  
)  
  
plt.xticks(rotation=45)  
plt.xlabel('Category')  
plt.ylabel('Count')  
plt.title('Top 10 Most Frequent News Categories')  
plt.tight_layout()  
plt.show()
```



```
In [5]: df.isna().sum()
```

```
Out[5]: link          0
        headline      0
        category      0
        short_description  0
        authors        0
        date           0
        dtype: int64
```

```
In [6]: df = df[['headline', 'category']]
```

```
In [7]: df
```

```
Out[7]:
```

	headline	category
0	Over 4 Million Americans Roll Up Sleeves For O...	U.S. NEWS
1	American Airlines Flyer Charged, Banned For Li...	U.S. NEWS
2	23 Of The Funniest Tweets About Cats And Dogs ...	COMEDY
3	The Funniest Tweets From Parents This Week (Se...	PARENTING
4	Woman Who Called Cops On Black Bird-Watcher Lo...	U.S. NEWS
...	...	...
209522	RIM CEO Thorsten Heins' 'Significant' Plans Fo...	TECH
209523	Maria Sharapova Stunned By Victoria Azarenka I...	SPORTS
209524	Giants Over Patriots, Jets Over Colts Among M...	SPORTS
209525	Aldon Smith Arrested: 49ers Linebacker Busted ...	SPORTS
209526	Dwight Howard Rips Teammates After Magic Loss ...	SPORTS

209527 rows × 2 columns

```
In [8]: for h in df['headline'].iloc[-3:-1]:
        print(h)
```

Giants Over Patriots, Jets Over Colts Among Most Improbable Super Bowl Upsets Of All Time (VIDEOS)
Aldon Smith Arrested: 49ers Linebacker Busted For DUI

```
In [9]: contents = df['headline'][2]
```

```
In [10]: from nltk.tokenize import word_tokenize
         toks = word_tokenize(contents)
         print(toks[:10])
```

```
['23', 'Of', 'The', 'Funniest', 'Tweets', 'About', 'Cats', 'And', 'Dogs', 'This']
```

```
In [11]: from nltk.corpus import stopwords
         stops = stopwords.words('english')
         nonstops = [t for t in toks if t.lower() not in stops]
         print(f"There are {len(nonstops)} non-stop word tokens in this article.")
```

```
There are 10 non-stop word tokens in this article.
```

```
In [12]: import spacy
         nlp = spacy.load("en_core_web_lg")
```

```
In [13]: document = nlp(contents)
         document
```

```
Out[13]: 23 Of The Funniest Tweets About Cats And Dogs This Week (Sept. 17-23)
```

```
In [14]: import re
         import string

         from nltk.corpus import stopwords
         from nltk.stem import WordNetLemmatizer
         from nltk.tokenize import word_tokenize
```

```
In [15]: stop_words = set(stopwords.words("english"))
         lemmatizer = WordNetLemmatizer()

         def preprocess(text):

             text = text.lower()

             text = re.sub(f'[{re.escape(string.punctuation)}]', ' ', text)

             tokens = word_tokenize(text)
```

```
tokens = [word for word in tokens if word.isalpha() and word not in stop_words]

tokens = [lemmatizer.lemmatize(word) for word in tokens]

return ' '.join(tokens)
```

```
In [16]: df['Clean_Headline'] = df['headline'].apply(preprocess)
```

```
In [17]: from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [18]: vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['Clean_Headline'])
print(X.shape)
```

(209527, 49250)

```
In [19]: vectorizer = TfidfVectorizer(
    lowercase=False,
    max_features= 20000,
    min_df=5,
    max_df=0.9,
    ngram_range=(1,2)
)

X = vectorizer.fit_transform(df['Clean_Headline'])
```

```
In [20]: print(len(vectorizer.get_feature_names_out()))
```

20000

```
In [21]: feature_names = vectorizer.get_feature_names_out()

print(feature_names[:50])
```

['aap' 'aaron' 'aaron carter' ... 'young men' 'young people' 'young woman']

```
In [22]: import pandas as pd

tfidf_df = pd.DataFrame(
    X[:5,:10].toarray(),
```

```

        columns=feature_names[:10]
    )
    tfidf_df

```

Out[22]:

	aap	aaron	aaron carter	aaron paul	aaron rodgers	aaron schock	ab	abandon	abandoned	abandoning
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

```

In [23]: from sklearn.model_selection import train_test_split

y = df['category']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

```

```

In [24]: print(X_train.shape)
         print(X_test.shape)

```

```

(167621, 20000)
(41906, 20000)

```

```

In [45]: from sklearn.naive_bayes import MultinomialNB

param_grid = {
    'alpha': [0.1, 0.5, 1.0, 2.0]
}

```

```
grid_nb = GridSearchCV(  
    estimator=MultinomialNB(),  
    param_grid=param_grid,  
    cv=5,  
    scoring='f1_macro',  
    n_jobs=-1  
)  
  
# Train  
grid_nb.fit(X_train, y_train)  
  
# Best model  
best_nb = grid_nb.best_estimator_  
  
# Predict  
y_pred_nb = best_nb.predict(X_test)  
  
print("Best Parameters:", grid_nb.best_params_)  
print("Best Cross Validation Macro F1:", grid_nb.best_score_)  
  
print("Accuracy:", accuracy_score(y_test, y_pred_nb))  
print(classification_report(y_test, y_pred_nb))
```

Best Parameters: {'alpha': 0.1}

Best Cross Validation Macro F1: 0.3807999423855857

Accuracy: 0.5493724049062186

	precision	recall	f1-score	support
ARTS	0.46	0.15	0.23	302
ARTS & CULTURE	0.58	0.11	0.19	268
BLACK VOICES	0.53	0.34	0.41	917
BUSINESS	0.47	0.39	0.43	1198
COLLEGE	0.50	0.18	0.27	229
COMEDY	0.58	0.39	0.46	1080
CRIME	0.49	0.58	0.53	712
CULTURE & ARTS	0.67	0.18	0.29	215
DIVORCE	0.79	0.56	0.66	685
EDUCATION	0.62	0.13	0.21	203
ENTERTAINMENT	0.54	0.73	0.62	3473
ENVIRONMENT	0.62	0.17	0.27	289
FIFTY	0.61	0.04	0.07	280
FOOD & DRINK	0.59	0.68	0.63	1268
GOOD NEWS	0.39	0.15	0.21	280
GREEN	0.46	0.30	0.37	524
HEALTHY LIVING	0.28	0.12	0.17	1339
HOME & LIVING	0.70	0.60	0.65	864
IMPACT	0.40	0.23	0.29	697
LATINO VOICES	0.68	0.09	0.16	226
MEDIA	0.62	0.30	0.40	589
MONEY	0.52	0.23	0.31	351
PARENTING	0.44	0.53	0.48	1758
PARENTS	0.41	0.18	0.25	791
POLITICS	0.58	0.87	0.70	7121
QUEER VOICES	0.69	0.57	0.62	1269
RELIGION	0.66	0.32	0.43	515
SCIENCE	0.70	0.39	0.50	441
SPORTS	0.67	0.65	0.66	1015
STYLE	0.59	0.12	0.20	451
STYLE & BEAUTY	0.67	0.76	0.71	1963
TASTE	0.34	0.07	0.12	419
TECH	0.61	0.32	0.42	421
THE WORLDPOST	0.46	0.41	0.44	733
TRAVEL	0.59	0.73	0.65	1980
U.S. NEWS	0.38	0.06	0.11	275
WEDDINGS	0.82	0.62	0.71	731

WEIRD NEWS	0.37	0.21	0.27	555
WELLNESS	0.46	0.73	0.56	3589
WOMEN	0.45	0.22	0.30	714
WORLD NEWS	0.43	0.29	0.35	660
WORLDPOST	0.35	0.14	0.20	516
accuracy			0.55	41906
macro avg	0.54	0.35	0.39	41906
weighted avg	0.54	0.55	0.52	41906

In [26]: *# Precision: When the model predicts SPORTS, how often is it correct?*
Recall: Of all actual SPORTS headlines, how many did the model find?
#F1 score: F1 is high only if both precision and recall are reasonably high.

In [28]: `from sklearn.metrics import accuracy_score`
`from sklearn.metrics import classification_report`
`from sklearn.metrics import confusion_matrix`

`print("Accuracy:", accuracy_score(y_test, y_pred_nb))`

`print(classification_report(y_test, y_pred_nb))`

Accuracy: 0.49565694649930797

C:\Users\Administrator\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
 _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

	precision	recall	f1-score	support
ARTS	0.50	0.01	0.02	302
ARTS & CULTURE	0.00	0.00	0.00	268
BLACK VOICES	0.66	0.17	0.27	917
BUSINESS	0.55	0.22	0.31	1198
COLLEGE	0.67	0.01	0.02	229
COMEDY	0.72	0.24	0.36	1080
CRIME	0.62	0.37	0.47	712
CULTURE & ARTS	1.00	0.07	0.14	215
DIVORCE	0.90	0.37	0.53	685
EDUCATION	0.80	0.02	0.04	203
ENTERTAINMENT	0.47	0.78	0.59	3473
ENVIRONMENT	0.97	0.10	0.18	289
FIFTY	0.00	0.00	0.00	280
FOOD & DRINK	0.63	0.63	0.63	1268
GOOD NEWS	0.83	0.02	0.03	280
GREEN	0.49	0.07	0.12	524
HEALTHY LIVING	0.47	0.03	0.05	1339
HOME & LIVING	0.83	0.48	0.61	864
IMPACT	0.66	0.05	0.10	697
LATINO VOICES	0.00	0.00	0.00	226
MEDIA	0.69	0.08	0.14	589
MONEY	0.86	0.05	0.10	351
PARENTING	0.46	0.50	0.48	1758
PARENTS	0.64	0.04	0.07	791
POLITICS	0.44	0.95	0.60	7121
QUEER VOICES	0.80	0.46	0.58	1269
RELIGION	0.82	0.14	0.24	515
SCIENCE	0.90	0.14	0.24	441
SPORTS	0.71	0.49	0.58	1015
STYLE	0.80	0.01	0.02	451
STYLE & BEAUTY	0.65	0.75	0.70	1963
TASTE	1.00	0.01	0.02	419
TECH	0.82	0.09	0.16	421
THE WORLDPOST	0.55	0.22	0.31	733
TRAVEL	0.57	0.72	0.63	1980
U.S. NEWS	0.00	0.00	0.00	275
WEDDINGS	0.90	0.49	0.63	731
WEIRD NEWS	0.66	0.03	0.07	555
WELLNESS	0.39	0.81	0.52	3589
WOMEN	0.77	0.09	0.16	714

WORLD NEWS	0.52	0.12	0.20	660
WORLDPOST	0.54	0.01	0.03	516
accuracy			0.50	41906
macro avg	0.62	0.23	0.26	41906
weighted avg	0.57	0.50	0.42	41906

C:\Users\Administrator\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

C:\Users\Administrator\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In []:

In [46]: `from sklearn.linear_model import LogisticRegression`

```
param_grid = {
    'C': [0.1, 1, 10]
}

grid_lr = GridSearchCV(
    estimator=LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    param_grid=param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)

# Train
grid_lr.fit(X_train, y_train)

# Best model
best_lr = grid_lr.best_estimator_
```

```
# Predict
y_pred_lr = best_lr.predict(X_test)

print("Best Parameters:", grid_lr.best_params_)
print("Best Cross Validation Macro F1:", grid_lr.best_score_)

print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

Best Parameters: {'C': 10}

Best Cross Validation Macro F1: 0.44080019347698196

Accuracy: 0.5741421276189567

	precision	recall	f1-score	support
ARTS	0.30	0.21	0.24	302
ARTS & CULTURE	0.36	0.20	0.26	268
BLACK VOICES	0.49	0.43	0.46	917
BUSINESS	0.45	0.45	0.45	1198
COLLEGE	0.49	0.39	0.44	229
COMEDY	0.50	0.43	0.46	1080
CRIME	0.54	0.52	0.53	712
CULTURE & ARTS	0.47	0.28	0.35	215
DIVORCE	0.77	0.67	0.72	685
EDUCATION	0.41	0.29	0.34	203
ENTERTAINMENT	0.63	0.72	0.67	3473
ENVIRONMENT	0.40	0.27	0.32	289
FIFTY	0.29	0.16	0.20	280
FOOD & DRINK	0.63	0.65	0.64	1268
GOOD NEWS	0.36	0.23	0.28	280
GREEN	0.37	0.33	0.35	524
HEALTHY LIVING	0.27	0.22	0.24	1339
HOME & LIVING	0.71	0.67	0.69	864
IMPACT	0.32	0.27	0.29	697
LATINO VOICES	0.53	0.32	0.40	226
MEDIA	0.55	0.42	0.48	589
MONEY	0.44	0.32	0.38	351
PARENTING	0.49	0.55	0.52	1758
PARENTS	0.36	0.28	0.32	791
POLITICS	0.71	0.81	0.76	7121
QUEER VOICES	0.72	0.66	0.69	1269
RELIGION	0.58	0.48	0.52	515
SCIENCE	0.57	0.45	0.51	441
SPORTS	0.66	0.66	0.66	1015
STYLE	0.42	0.27	0.33	451
STYLE & BEAUTY	0.73	0.77	0.75	1963
TASTE	0.32	0.21	0.25	419
TECH	0.51	0.46	0.48	421
THE WORLDPOST	0.45	0.42	0.43	733
TRAVEL	0.65	0.74	0.69	1980
U.S. NEWS	0.33	0.14	0.20	275
WEDDINGS	0.80	0.75	0.77	731

WEIRD NEWS	0.31	0.26	0.29	555
WELLNESS	0.50	0.67	0.57	3589
WOMEN	0.38	0.32	0.35	714
WORLD NEWS	0.42	0.38	0.40	660
WORLDPOST	0.33	0.25	0.28	516
accuracy			0.57	41906
macro avg	0.49	0.43	0.45	41906
weighted avg	0.56	0.57	0.56	41906

```
In [31]: print(classification_report(y_test, y_pred_lr))
```

	precision	recall	f1-score	support
ARTS	0.40	0.18	0.25	302
ARTS & CULTURE	0.41	0.14	0.21	268
BLACK VOICES	0.56	0.38	0.45	917
BUSINESS	0.50	0.45	0.47	1198
COLLEGE	0.56	0.36	0.44	229
COMEDY	0.63	0.41	0.50	1080
CRIME	0.55	0.53	0.54	712
CULTURE & ARTS	0.73	0.21	0.32	215
DIVORCE	0.84	0.62	0.72	685
EDUCATION	0.44	0.24	0.31	203
ENTERTAINMENT	0.57	0.77	0.65	3473
ENVIRONMENT	0.64	0.19	0.29	289
FIFTY	0.56	0.12	0.20	280
FOOD & DRINK	0.62	0.69	0.66	1268
GOOD NEWS	0.46	0.13	0.20	280
GREEN	0.45	0.33	0.38	524
HEALTHY LIVING	0.36	0.16	0.22	1339
HOME & LIVING	0.71	0.67	0.69	864
IMPACT	0.45	0.25	0.32	697
LATINO VOICES	0.71	0.24	0.36	226
MEDIA	0.61	0.38	0.47	589
MONEY	0.52	0.26	0.34	351
PARENTING	0.49	0.63	0.55	1758
PARENTS	0.48	0.20	0.28	791
POLITICS	0.65	0.86	0.74	7121
QUEER VOICES	0.78	0.65	0.71	1269
RELIGION	0.63	0.44	0.52	515
SCIENCE	0.66	0.43	0.52	441
SPORTS	0.67	0.66	0.67	1015
STYLE	0.52	0.14	0.22	451
STYLE & BEAUTY	0.70	0.78	0.74	1963
TASTE	0.41	0.11	0.18	419
TECH	0.56	0.41	0.47	421
THE WORLDPOST	0.49	0.39	0.44	733
TRAVEL	0.63	0.76	0.69	1980
U.S. NEWS	0.48	0.05	0.10	275
WEDDINGS	0.80	0.73	0.76	731
WEIRD NEWS	0.38	0.20	0.26	555
WELLNESS	0.45	0.76	0.57	3589
WOMEN	0.45	0.29	0.35	714

WORLD NEWS	0.48	0.32	0.38	660
WORLDPOST	0.41	0.19	0.26	516
accuracy			0.58	41906
macro avg	0.56	0.40	0.44	41906
weighted avg	0.58	0.58	0.56	41906

```
In [44]: from sklearn.svm import LinearSVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report

param_grid = {
    'C': [0.1, 1, 10]
}

grid = GridSearchCV(
    estimator=LinearSVC(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)

# Train and tune
grid.fit(X_train, y_train)

# Best model
best_svm = grid.best_estimator_

# Predictions using the best model
y_pred_svm = best_svm.predict(X_test)

print("Best Parameters:", grid.best_params_)
print("Best Cross Validation Macro F1:", grid.best_score_)

print("Test Accuracy:", accuracy_score(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

Best Parameters: {'C': 1}

Best Cross Validation Macro F1: 0.4399166670143659

Test Accuracy: 0.5821839354746337

	precision	recall	f1-score	support
ARTS	0.30	0.22	0.25	302
ARTS & CULTURE	0.35	0.19	0.25	268
BLACK VOICES	0.51	0.43	0.47	917
BUSINESS	0.46	0.45	0.46	1198
COLLEGE	0.47	0.43	0.45	229
COMEDY	0.55	0.42	0.48	1080
CRIME	0.52	0.55	0.53	712
CULTURE & ARTS	0.45	0.25	0.32	215
DIVORCE	0.76	0.68	0.72	685
EDUCATION	0.40	0.33	0.37	203
ENTERTAINMENT	0.64	0.73	0.68	3473
ENVIRONMENT	0.39	0.25	0.30	289
FIFTY	0.33	0.18	0.23	280
FOOD & DRINK	0.60	0.68	0.64	1268
GOOD NEWS	0.35	0.21	0.26	280
GREEN	0.37	0.33	0.35	524
HEALTHY LIVING	0.30	0.18	0.22	1339
HOME & LIVING	0.67	0.69	0.68	864
IMPACT	0.34	0.25	0.29	697
LATINO VOICES	0.57	0.35	0.44	226
MEDIA	0.55	0.44	0.49	589
MONEY	0.46	0.34	0.39	351
PARENTING	0.49	0.56	0.52	1758
PARENTS	0.38	0.24	0.29	791
POLITICS	0.72	0.82	0.77	7121
QUEER VOICES	0.74	0.69	0.71	1269
RELIGION	0.56	0.50	0.53	515
SCIENCE	0.55	0.48	0.51	441
SPORTS	0.62	0.69	0.65	1015
STYLE	0.42	0.21	0.28	451
STYLE & BEAUTY	0.71	0.79	0.75	1963
TASTE	0.28	0.15	0.19	419
TECH	0.50	0.49	0.49	421
THE WORLDPOST	0.45	0.42	0.43	733
TRAVEL	0.65	0.76	0.70	1980
U.S. NEWS	0.30	0.10	0.15	275
WEDDINGS	0.77	0.78	0.78	731

WEIRD NEWS	0.33	0.24	0.28	555
WELLNESS	0.51	0.70	0.59	3589
WOMEN	0.39	0.30	0.34	714
WORLD NEWS	0.44	0.36	0.39	660
WORLDPOST	0.35	0.25	0.29	516
accuracy			0.58	41906
macro avg	0.49	0.43	0.45	41906
weighted avg	0.56	0.58	0.57	41906

```
In [48]: import time

start = time.time()

best_lr.fit(X_train, y_train)

lr_train_time = time.time() - start

start = time.time()

y_pred_lr = best_lr.predict(X_test)

lr_predict_time = time.time() - start

print(f"Training Time: {lr_train_time:.2f} seconds")
print(f"Prediction Time: {lr_predict_time:.2f} seconds")
```

Training Time: 207.61 seconds
Prediction Time: 0.04 seconds

```
In [49]: import time

start = time.time()

best_nb.fit(X_train, y_train)

nb_train_time = time.time() - start

start = time.time()

y_pred_nb = best_nb.predict(X_test)
```

```
nb_predict_time = time.time() - start

print(f"Training Time: {nb_train_time:.2f} seconds")
print(f"Prediction Time: {nb_predict_time:.2f} seconds")
```

Training Time: 2.25 seconds
Prediction Time: 0.04 seconds

```
In [33]: import time

start = time.time()

svm = LinearSVC(random_state=42)
svm.fit(X_train, y_train)

train_time = time.time() - start

print(train_time)
```

55.032222509384155

```
In [34]: start = time.time()

y_pred_svm = svm.predict(X_test)

predict_time = time.time() - start

print(predict_time)
```

0.04564809799194336

```
In [35]: import numpy as np

feature_names = vectorizer.get_feature_names_out()

category = "POLITICS"

# Find the index of the category
idx = np.where(lr.classes_ == category)[0][0]

# Coefficients for that category
coef = lr.coef_[idx]
```

```
# Top 20 positive features
top_idx = np.argsort(coef)[-20:]

print(f"Top words for {category}:\n")

for i in reversed(top_idx):
    print(f"{feature_names[i]:25} {coef[i]:.4f}")
```

Top words for POLITICS:

trump	13.6709
democrat	8.1420
gop	7.9945
obama	7.7987
republican	7.1232
senate	7.0260
clinton	6.4043
biden	6.3927
congress	6.2607
senator	6.1441
democratic	5.5188
congressman	5.4877
morning email	5.4155
huffpollster	5.4009
rubio	5.3209
obamacare	5.2539
lawmaker	5.0340
sander	4.9590
election	4.9150
huffpost hill	4.8938

```
In [36]: categories = [
    "POLITICS",
    "SPORTS",
    "BUSINESS",
    "TECH",
    "ENTERTAINMENT"
]

feature_names = vectorizer.get_feature_names_out()
```

```
for category in categories:

    idx = np.where(lr.classes_ == category)[0][0]
    coef = lr.coef_[idx]
    top_idx = np.argsort(coef)[-10:]

    print("="*60)
    print(category)
    print("="*60)

    for i in reversed(top_idx):
        print(f"{feature_names[i]:20} {coef[i]:.4f}")

    print()
```

=====

POLITICS

=====

trump	13.6709
democrat	8.1420
gop	7.9945
obama	7.7987
republican	7.1232
senate	7.0260
clinton	6.4043
biden	6.3927
congress	6.2607
senator	6.1441

=====

SPORTS

=====

nfl	8.6294
nba	7.2042
sport	5.8759
football	5.8107
mlb	5.7436
lebron	5.5715
fan	5.5440
soccer	5.5164
olympic	5.3575
player	5.1345

=====

BUSINESS

=====

business	6.2283
uber	5.6718
woman business	5.4646
ceo	5.4236
krugman	5.1780
wall st	5.0970
marketing	5.0218
worker	4.8480
job	4.6464
company	4.6097

```
=====
TECH
=====
apple            8.1638
google           8.1590
facebook         7.7202
tech             6.2637
iphone           6.1804
microsoft        5.8178
twitter          5.5050
hacker           4.9416
samsung          4.6568
amazon           4.5753

=====
ENTERTAINMENT
=====
trailer          7.2805
box office       6.5999
movie            5.9715
kardashian       5.3434
netflix          5.2332
snl              4.7920
oscar            4.7362
adele            4.5646
clip             4.5083
jenner           4.3434
```

```
In [37]: import pandas as pd
import numpy as np

feature_names = vectorizer.get_feature_names_out()

top_words = {}

for category in lr.classes_:

    idx = np.where(lr.classes_ == category)[0][0]
    coef = lr.coef_[idx]

    top_idx = np.argsort(coef)[-10:]
```

```

top_words[category] = feature_names[top_idx][::-1]

top_df = pd.DataFrame(top_words)

top_df

```

Out[37]:

	ARTS	ARTS & CULTURE	BLACK VOICES	BUSINESS	COLLEGE	COMEDY	CRIME	CULTURE & ARTS	DIVORCE	EDUCATION	..
0	art	artist	black	business	college	snl	cop	photo	divorce	education	.
1	artist	book	racial	uber	university	colbert	police	imageblog	divorced	school	.
2	photography	art	mlk	woman business	campus	trump	shooting	art	ex	teacher	.
3	photographer	broadway	racist	ceo	student	conan	accused	artist	marriage	teaching	.
4	theater	hamilton	cop	krugman	frat	maher	allegedly	gallery	breakup	student	.
5	opera	photographer	african american	wall st	fraternity	comedian	prison	week art	divorcing	devos	.
6	first nighter	rowling	racism	marketing	professor	video	murder	poetry	cheating	learning	.
7	nighter	feminist	police	worker	grad	john oliver	shooter	painting	affair	classroom	.
8	portrait	author	white	job	commencement	spoof	man	exhibition	relationship	college	.
9	ballet	museum	african	company	education	comedy	suspect	theatre	single	charter	.

10 rows × 42 columns



```

In [38]: results = pd.DataFrame({
    'Model': ['Naive Bayes', 'Logistic Regression', 'Linear SVM'],
    'Accuracy': [0.50, 0.58, 0.582],
    'Macro Precision': [0.62, 0.56, 0.49],
    'Macro Recall': [0.23, 0.40, 0.43],
    'Macro F1': [0.26, 0.44, 0.45]
})

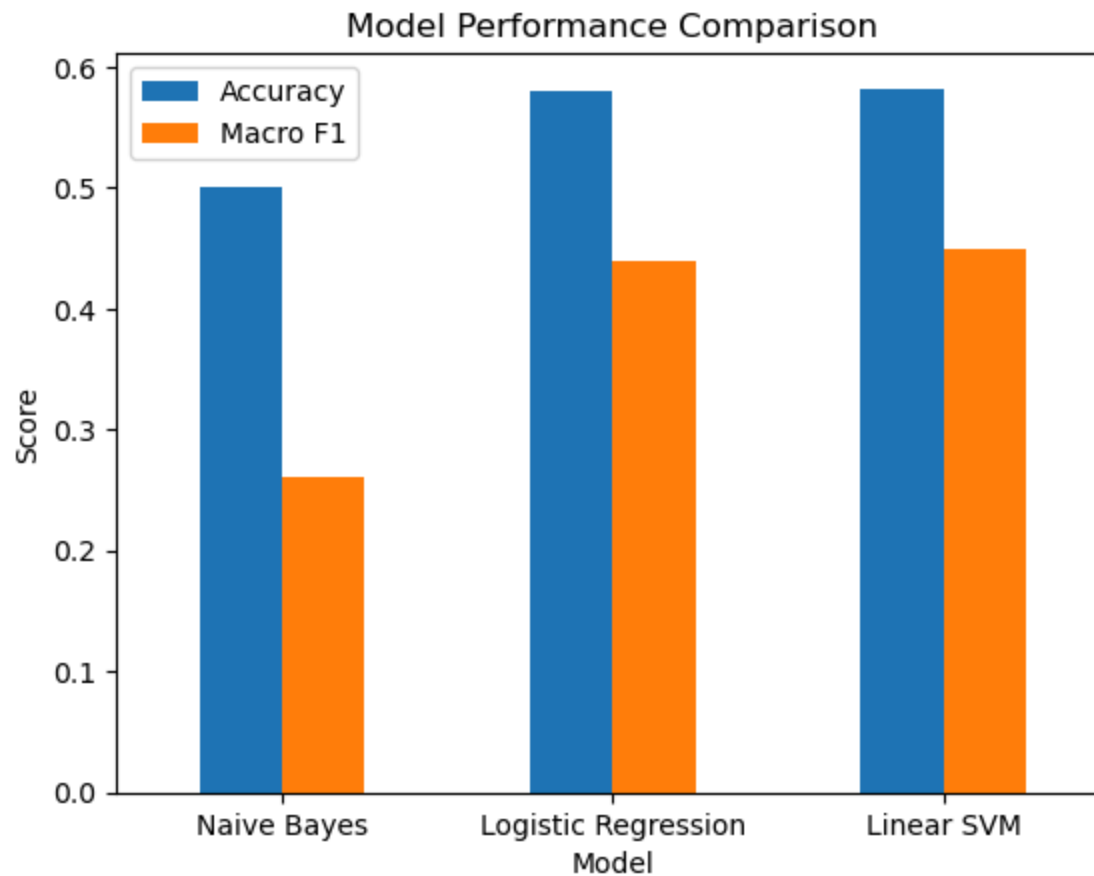
```

```
print(results)
```

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1
0	Naive Bayes	0.500	0.62	0.23	0.26
1	Logistic Regression	0.580	0.56	0.40	0.44
2	Linear SVM	0.582	0.49	0.43	0.45

```
In [39]: import matplotlib.pyplot as plt
```

```
results.plot(  
    x='Model',  
    y=['Accuracy', 'Macro F1'],  
    kind='bar',  
    rot=0  
)  
  
plt.ylabel("Score")  
plt.title("Model Performance Comparison")  
plt.show()
```

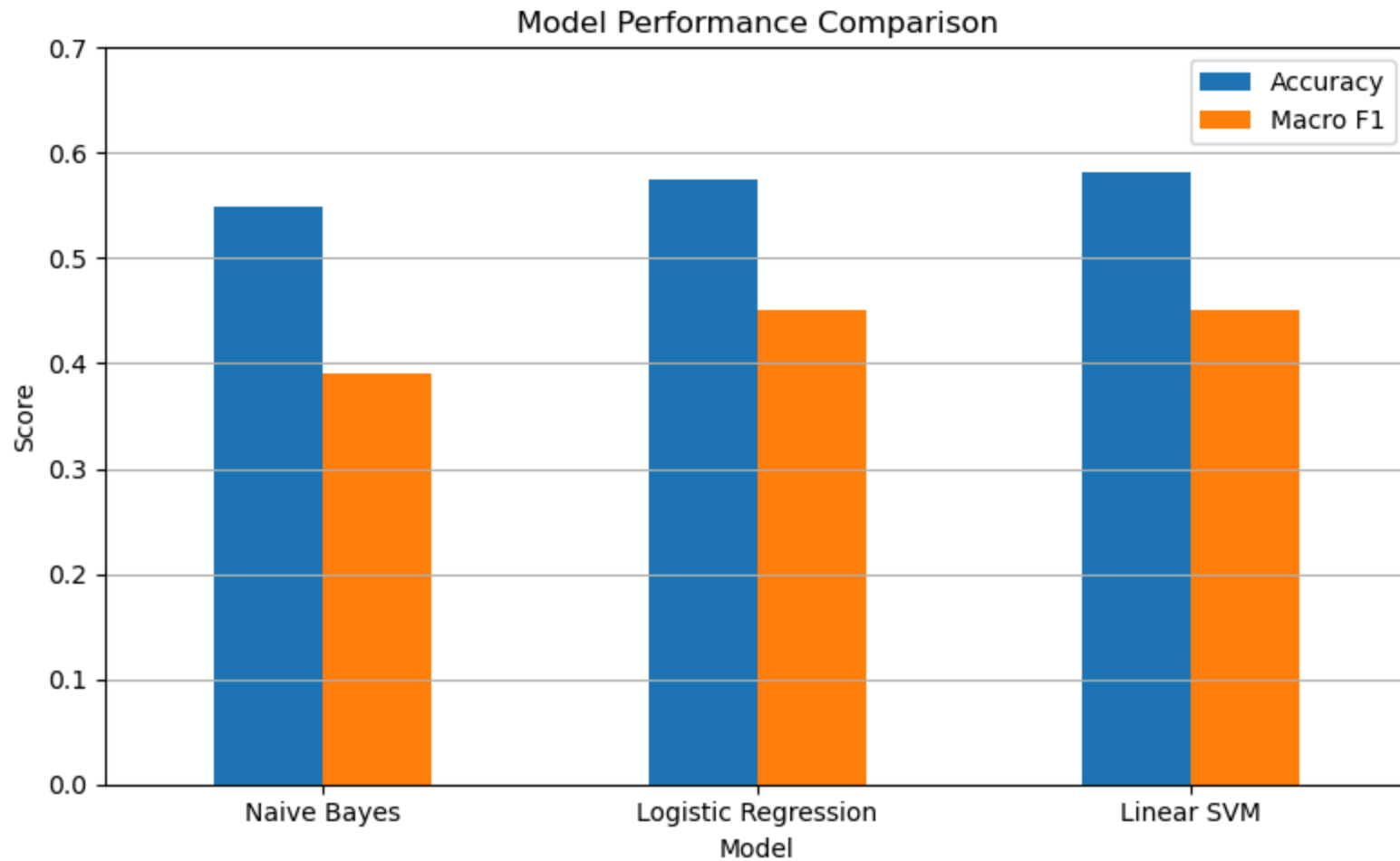


```
In [50]: import pandas as pd
import matplotlib.pyplot as plt

results = pd.DataFrame({
    'Model': ['Naive Bayes', 'Logistic Regression', 'Linear SVM'],
    'Accuracy': [0.549, 0.574, 0.582],
    'Macro F1': [0.39, 0.45, 0.45]
})

results.plot(
    x='Model',
    y=['Accuracy', 'Macro F1'],
    kind='bar',
    figsize=(8,5))
```

```
)  
  
plt.title('Model Performance Comparison')  
plt.ylabel('Score')  
plt.ylim(0,0.7)  
plt.xticks(rotation=0)  
plt.grid(axis='y')  
plt.tight_layout()  
plt.show()
```

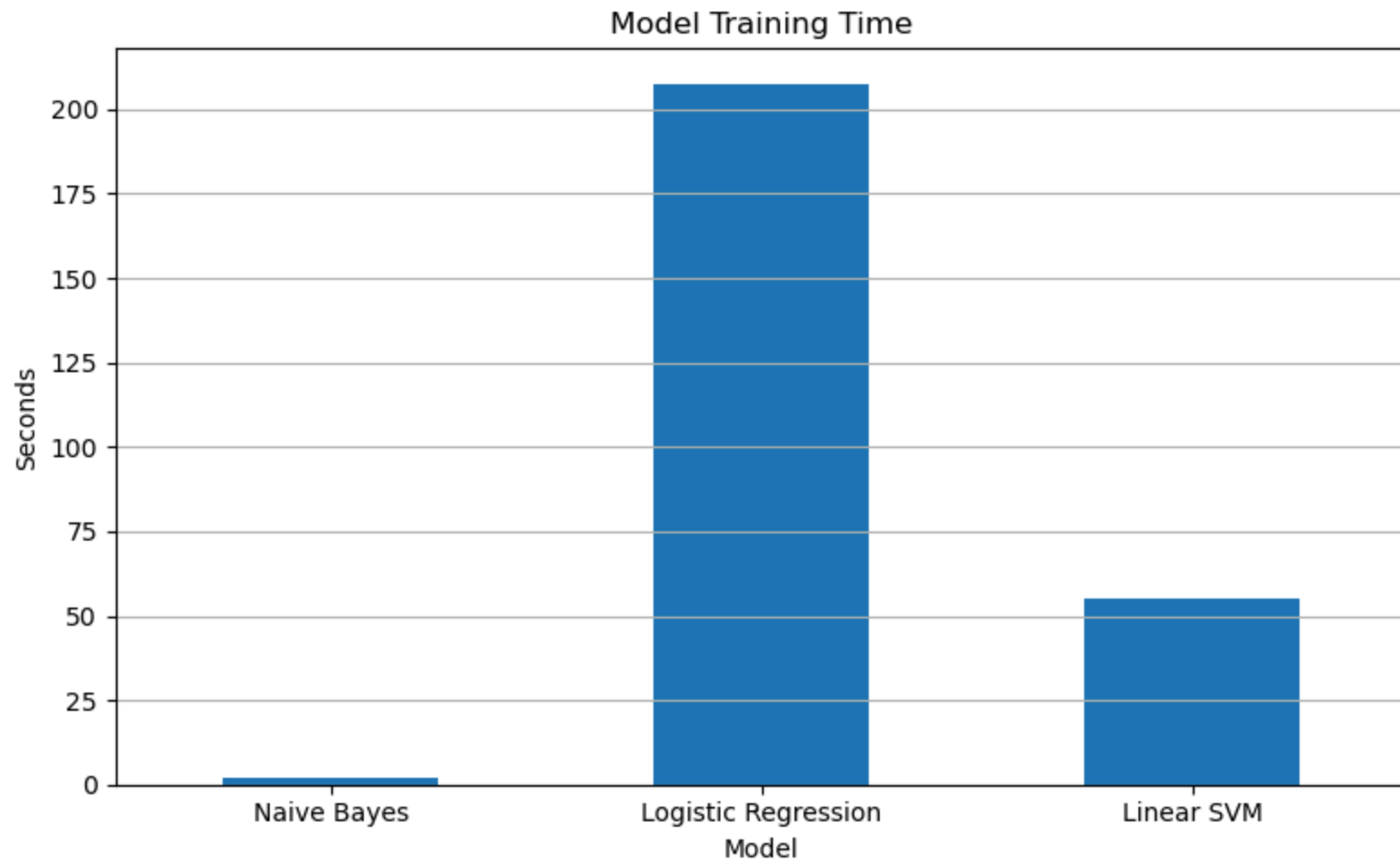


```
In [51]: import pandas as pd  
import matplotlib.pyplot as plt
```

```
times = pd.DataFrame({
    'Model': ['Naive Bayes', 'Logistic Regression', 'Linear SVM'],
    'Training Time (s)': [2.25, 207.61, 55.03]
})

times.plot(
    x='Model',
    y='Training Time (s)',
    kind='bar',
    figsize=(8,5),
    legend=False
)

plt.title('Model Training Time')
plt.ylabel('Seconds')
plt.xticks(rotation=0)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

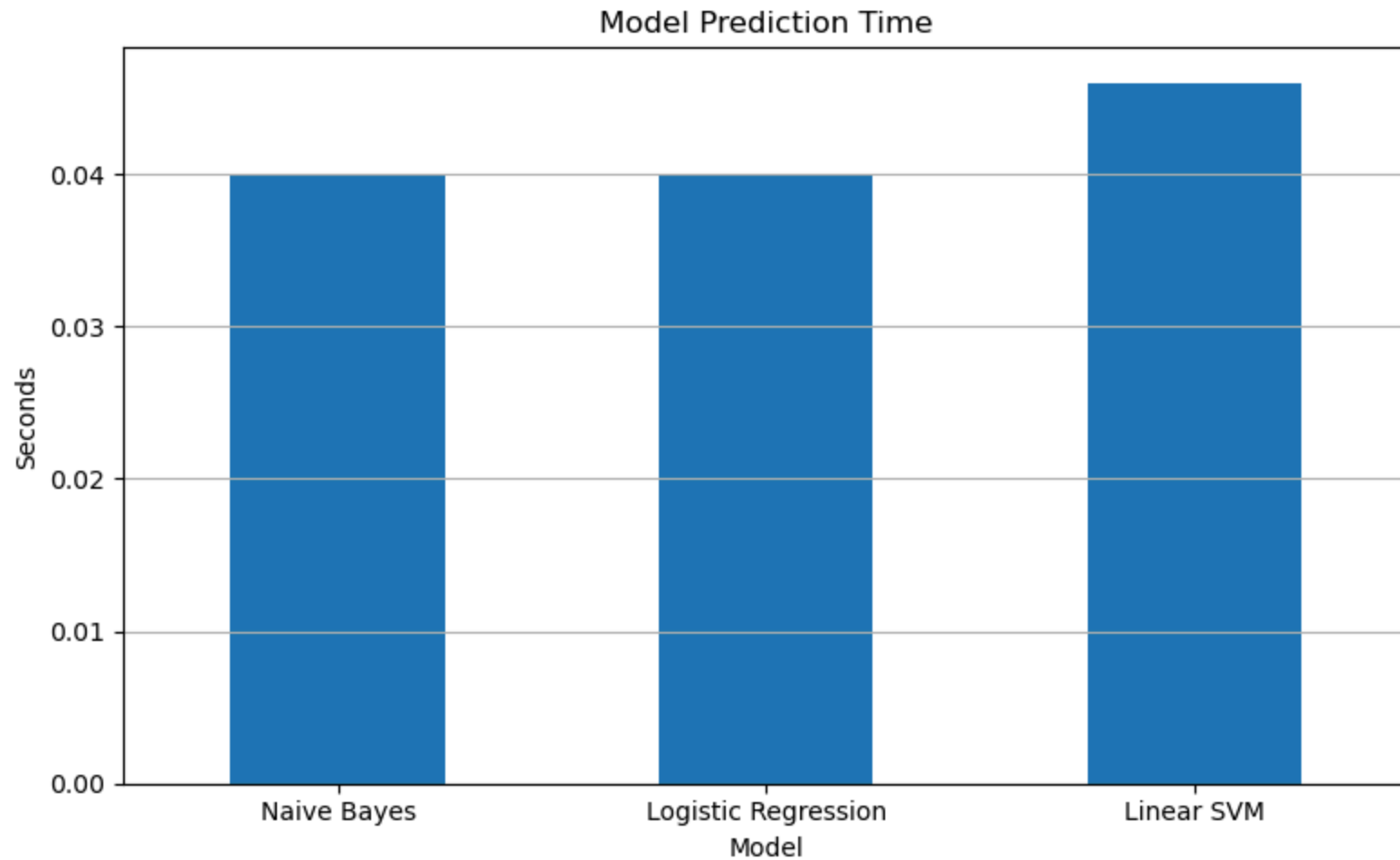


```
In [52]: import pandas as pd
import matplotlib.pyplot as plt

predict = pd.DataFrame({
    'Model': ['Naive Bayes', 'Logistic Regression', 'Linear SVM'],
    'Prediction Time (s)': [0.040, 0.040, 0.046]
})

predict.plot(
    x='Model',
    y='Prediction Time (s)',
    kind='bar',
```

```
figsize=(8,5),  
legend=False  
)  
  
plt.title('Model Prediction Time')  
plt.ylabel('Seconds')  
plt.xticks(rotation=0)  
plt.grid(axis='y')  
plt.tight_layout()  
plt.show()
```



```
In [53]: summary = pd.DataFrame({
    'Model': ['Naive Bayes', 'Logistic Regression', 'Linear SVM'],
    'Best Parameter': ['α=0.1', 'C=10', 'C=1'],
    'CV Macro F1': [0.381, 0.441, 0.440],
    'Accuracy': [0.549, 0.574, 0.582],
    'Macro F1': [0.39, 0.45, 0.45],
    'Train Time (s)': [2.25, 207.61, 55.03],
    'Predict Time (s)': [0.040, 0.040, 0.046]
})

print(summary)
```

	Model	Best Parameter	CV Macro F1	Accuracy	Macro F1	\
0	Naive Bayes	α=0.1	0.381	0.549	0.39	
1	Logistic Regression	C=10	0.441	0.574	0.45	
2	Linear SVM	C=1	0.440	0.582	0.45	

	Train Time (s)	Predict Time (s)
0	2.25	0.040
1	207.61	0.040
2	55.03	0.046

```
In [57]: import pandas as pd

results = pd.DataFrame({
    'Model': ['Naive Bayes', 'Logistic Regression', 'Linear SVM'],
    'Best Hyperparameter': ['α = 0.1', 'C = 10', 'C = 1'],
    'Accuracy': [0.50, 0.58, 0.582],
    'Macro Precision': [0.62, 0.56, 0.49],
    'Macro Recall': [0.23, 0.40, 0.43],
    'Macro F1': [0.26, 0.44, 0.45],
    'Training Time (s)': [2.25, 207.61, 55.03],
    'Prediction Time (s)': [0.040, 0.040, 0.046]
})

print(results)
```

	Model	Best Hyperparameter	Accuracy	Macro Precision	\
0	Naive Bayes	$\alpha = 0.1$	0.500	0.62	
1	Logistic Regression	C = 10	0.580	0.56	
2	Linear SVM	C = 1	0.582	0.49	

	Macro Recall	Macro F1	Training Time (s)	Prediction Time (s)
0	0.23	0.26	2.25	0.040
1	0.40	0.44	207.61	0.040
2	0.43	0.45	55.03	0.046

In [58]: results

Out[58]:

	Model	Best Hyperparameter	Accuracy	Macro Precision	Macro Recall	Macro F1	Training Time (s)	Prediction Time (s)
0	Naive Bayes	$\alpha = 0.1$	0.500	0.62	0.23	0.26	2.25	0.040
1	Logistic Regression	C = 10	0.580	0.56	0.40	0.44	207.61	0.040
2	Linear SVM	C = 1	0.582	0.49	0.43	0.45	55.03	0.046

```
In [60]: import matplotlib.pyplot as plt

words = ['trump', 'democrat', 'gop', 'obama', 'republican',
         'senate', 'clinton', 'biden', 'congress', 'senator']

scores = [13.6709, 8.1420, 7.9945, 7.7987, 7.1232,
          7.0260, 6.4043, 6.3927, 6.2607, 6.1441]

plt.figure(figsize=(7,5))

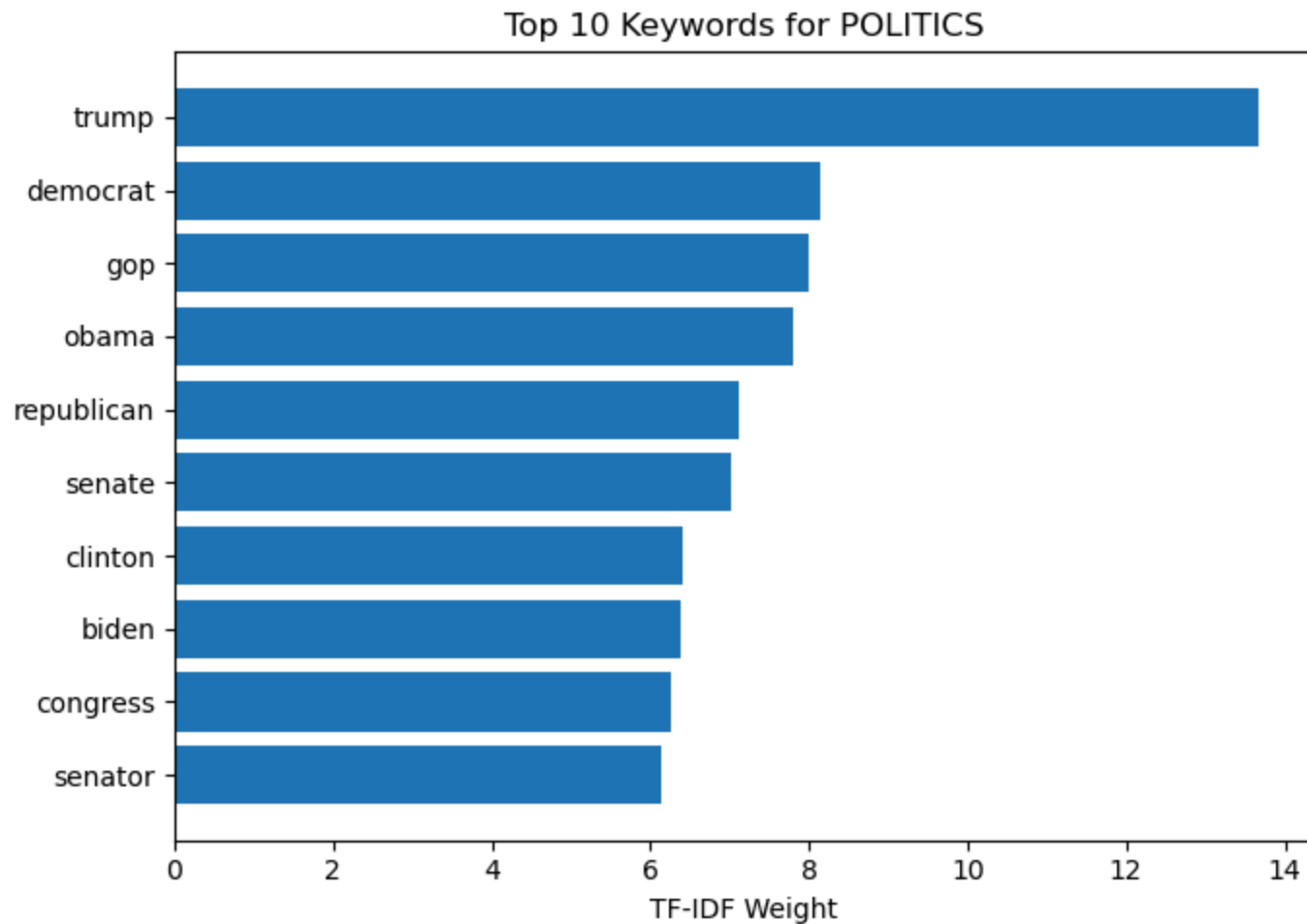
plt.barh(words, scores)

plt.gca().invert_yaxis()

plt.xlabel("TF-IDF Weight")

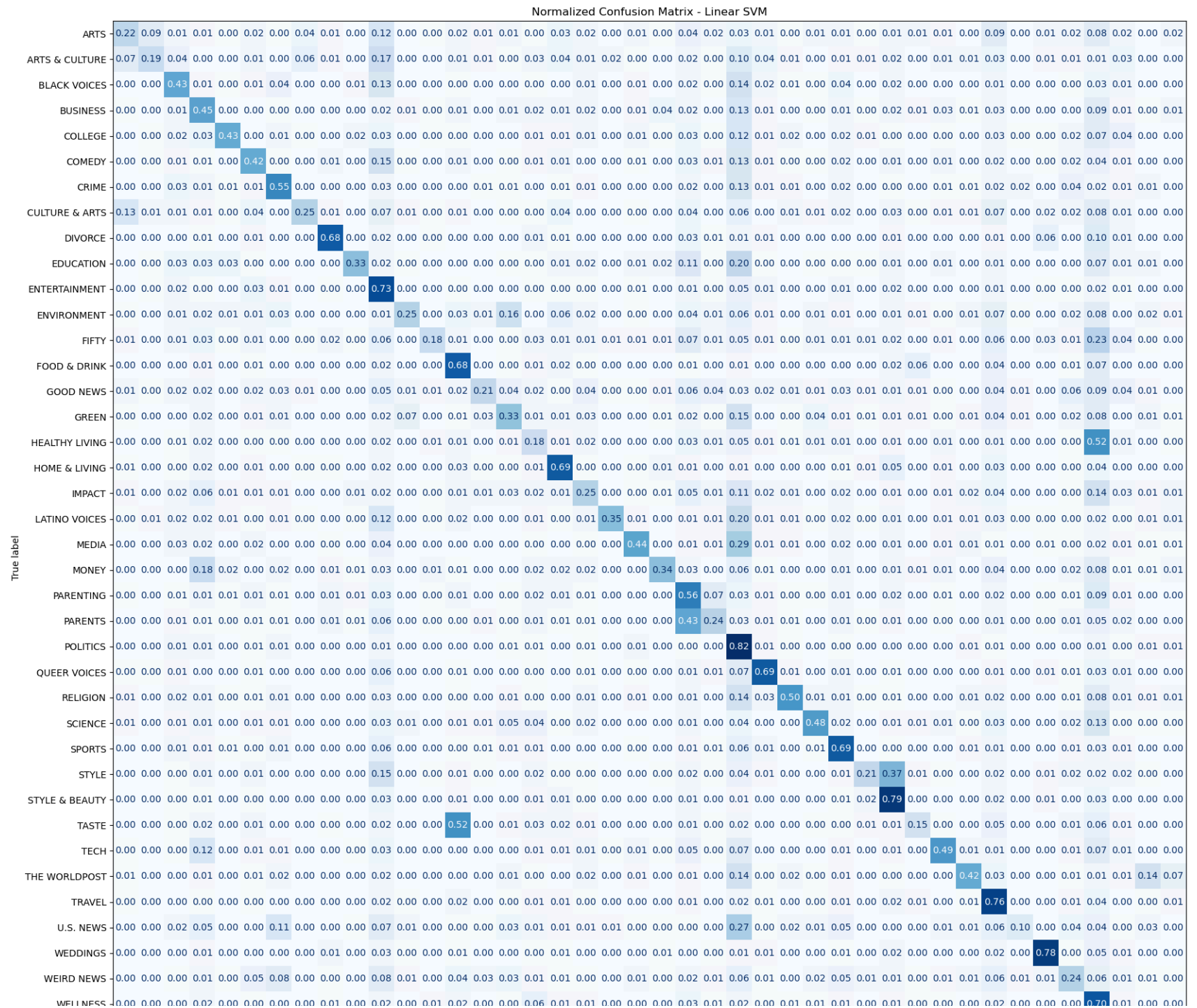
plt.title("Top 10 Keywords for POLITICS")
```

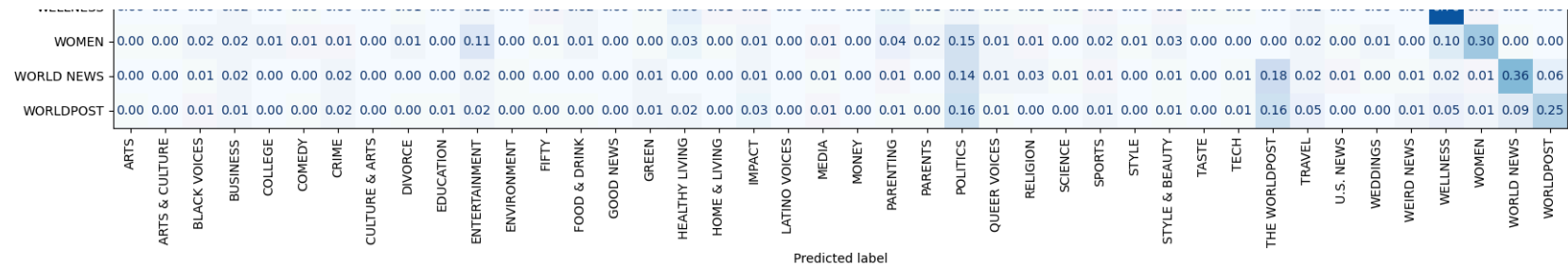
```
plt.tight_layout()  
plt.show()
```



```
In [63]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay  
import matplotlib.pyplot as plt  
  
cm = confusion_matrix(y_test, y_pred_svm, normalize='true')  
  
fig, ax = plt.subplots(figsize=(18, 18))  
  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=cm,
```

```
        display_labels=svm.classes_  
    )  
  
    disp.plot(  
        cmap='Blues',  
        ax=ax,  
        colorbar=False,  
        xticks_rotation=90,  
        values_format='.2f'  
    )  
  
    plt.title("Normalized Confusion Matrix - Linear SVM")  
    plt.tight_layout()  
    plt.show()
```





```
In [64]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_svm)

fig, ax = plt.subplots(figsize=(18,18))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=svm.classes_
)

disp.plot(
    cmap='Blues',
    ax=ax,
    colorbar=False,
    xticks_rotation=90
)

plt.title("Confusion Matrix - Linear SVM")
plt.tight_layout()
plt.show()
```

		Confusion Matrix - Linear SVM																																									
True label	ARTS	66	27	3	2	1	6	1	11	2	0	35	0	1	5	2	3	1	9	5	0	3	1	12	5	10	3	1	4	4	1	3	3	2	0	27	0	4	6	23	5	0	5
	ARTS & CULTURE	19	51	12	0	0	3	0	16	2	0	46	0	0	3	2	1	7	10	4	5	1	0	5	1	26	10	2	0	2	2	6	0	4	3	8	0	2	2	4	9	0	0
	BLACK VOICES	2	2	398	7	4	8	37	1	2	8	122	1	2	4	1	1	4	1	12	0	6	1	22	3	127	15	5	3	34	1	22	0	1	1	12	1	2	4	28	11	1	0
	BUSINESS	4	1	12	536	3	3	4	0	2	3	27	8	5	13	3	14	27	13	22	1	10	44	23	3	152	8	3	4	14	4	18	8	31	6	33	0	1	3	104	10	5	13
	COLLEGE	0	1	5	8	98	0	3	1	0	5	7	0	1	1	0	0	2	2	3	0	3	1	7	1	28	2	4	0	4	2	1	0	1	0	6	1	1	4	17	9	0	0
	COMEDY	3	4	7	6	3	455	5	0	11	3	157	4	2	16	2	5	7	12	3	3	6	2	33	10	141	9	4	5	20	1	16	3	13	3	25	0	5	22	44	9	0	1
	CRIME	0	1	21	7	4	5	390	0	2	2	20	1	0	2	4	4	2	4	5	0	2	0	17	2	94	6	6	0	12	1	3	0	4	4	13	14	2	28	14	5	9	2
	CULTURE & ARTS	27	3	3	2	1	8	0	54	2	0	14	2	1	2	0	1	1	8	0	0	1	0	8	0	13	0	3	2	4	0	6	1	2	2	15	0	4	5	17	3	0	0
	DIVORCE	1	1	2	4	0	5	2	0	465	1	17	0	3	1	1	0	7	4	1	0	1	0	23	4	7	5	0	2	1	0	6	0	2	1	5	0	38	2	66	7	0	0
	EDUCATION	0	1	6	6	7	1	0	0	0	68	4	1	0	0	0	1	1	2	5	0	2	5	22	1	41	1	0	1	1	1	2	0	2	0	2	0	0	15	2	2	0	
	ENTERTAINMENT	8	14	74	15	4	114	24	1	17	1	2518	2	5	14	2	6	16	14	12	8	21	4	37	13	163	46	11	8	45	8	65	4	8	5	39	2	14	12	72	21	4	2
	ENVIRONMENT	1	0	3	7	3	4	10	0	0	0	4	71	0	9	4	46	0	17	7	0	1	0	11	4	16	2	0	3	2	0	3	2	1	3	19	1	0	5	22	0	6	2
	FIFTY	2	0	3	9	1	4	0	1	5	1	18	0	50	3	0	1	8	4	3	2	2	2	21	3	13	2	1	3	3	3	5	0	2	0	18	0	7	2	65	12	1	0
	FOOD & DRINK	1	2	2	11	2	8	0	2	4	1	20	1	1	859	1	6	11	30	6	0	4	3	7	3	11	0	2	2	3	1	26	73	4	1	56	0	1	7	90	2	1	3
	GOOD NEWS	4	1	5	6	0	5	7	2	0	0	15	3	2	5	59	11	5	1	11	1	1	2	18	12	8	6	3	2	8	2	3	2	1	0	11	2	0	18	25	11	2	0
	GREEN	2	0	1	12	1	4	6	1	0	0	11	35	0	3	15	174	6	6	14	0	1	3	11	2	80	1	2	19	4	3	4	4	2	4	23	5	1	12	42	0	7	3
	HEALTHY LIVING	3	3	8	24	2	5	6	2	3	1	23	1	9	18	1	9	237	9	23	1	3	2	44	13	64	9	8	14	14	3	11	4	17	3	19	1	3	6	690	15	3	5
	HOME & LIVING	5	0	3	15	2	6	1	3	4	0	21	0	0	25	3	3	7	593	3	0	3	11	8	2	8	1	1	1	5	7	45	2	6	1	28	1	1	4	32	1	1	1
	IMPACT	5	0	17	40	5	5	6	0	2	8	15	1	3	7	8	19	13	9	172	2	3	4	38	9	77	13	10	1	15	1	6	1	6	15	25	0	0	3	101	18	6	8
	LATINO VOICES	1	2	4	4	2	1	2	1	0	1	28	0	1	4	0	0	2	1	3	80	2	0	2	2	45	2	2	1	5	1	3	0	2	2	6	1	1	1	5	1	2	3
	MEDIA	1	0	15	9	1	14	1	0	2	1	25	1	1	1	0	1	2	1	2	1	260	1	5	5	170	5	4	1	11	1	6	0	5	8	3	0	3	2	9	5	3	3
	MONEY	0	0	0	64	8	1	7	0	3	2	10	1	5	2	0	0	7	6	6	0	0	119	10	0	21	2	0	1	5	1	4	3	4	0	15	0	1	6	29	2	2	4
	PARENTING	7	5	9	17	10	17	10	2	19	12	56	1	7	21	2	7	31	23	9	0	2	3	993	129	52	12	6	8	17	0	28	3	9	2	28	2	6	9	165	14	3	2
	PARENTS	2	2	7	5	3	5	4	2	10	5	48	0	3	3	7	3	11	4	5	1	1	2	342	189	20	6	4	1	8	3	6	0	3	0	11	0	3	9	39	14	0	0
	POLITICS	5	5	69	102	10	41	69	2	4	19	85	8	5	6	6	45	42	5	38	22	75	16	34	6	5870	51	35	13	25	1	12	2	21	62	43	14	3	11	96	35	48	60
	QUEER VOICES	6	5	12	2	1	8	4	1	2	2	72	0	2	7	0	3	6	1	8	2	6	0	18	10	92	870	11	1	13	4	8	1	4	2	9	1	10	7	38	16	4	0
	RELIGION	4	1	9	6	2	3	5	1	1	0	14	0	1	2	1	4	2	1	6	0	4	0	7	1	73	13	256	7	5	2	3	1	1	5	12	0	1	3	41	6	4	7
	SCIENCE	3	0	4	6	1	3	3	1	0	0	13	5	0	4	4	21	19	2	7	1	1	1	5	1	16	0	1	210	8	0	4	3	6	2	13	0	0	9	58	2	2	2
	SPORTS	2	0	11	9	6	4	11	1	1	2	60	1	2	4	6	7	7	2	3	1	4	1	9	6	57	14	3	6	697	3	4	1	1	7	12	3	2	7	26	7	3	2
	STYLE	0	0	2	6	0	5	0	1	1	0	67	0	2	4	2	0	9	2	2	0	0	2	7	2	19	4	0	0	4	96	169	4	2	0	8	0	3	7	10	9	2	0
STYLE & BEAUTY	3	3	9	11	2	7	2	1	3	0	67	0	3	13	1	1	15	27	7	0	2	5	19	5	18	3	0	2	12	48	1544	3	3	3	32	1	15	4	59	6	1	3	
TASTE	0	0	2	9	1	5	1	0	2	1	9	0	1	217	1	3	11	8	3	0	0	0	5	2	7	0	0	2	2	3	3	62	2	1	21	0	1	6	24	4	0	0	
TECH	2	0	2	51	1	4	3	0	0	2	13	1	0	0	1	0	4	4	5	2	6	2	19	2	30	1	1	1	3	0	3	1	205	5	6	0	0	4	31	5	1	0	
THE WORLDPOST	4	1	2	6	1	4	15	1	0	1	12	0	2	2	2	8	3	0	16	1	5	1	5	0	100	3	11	3	6	1	3	0	2	309	25	2	0	4	10	7	105	50	
TRAVEL	8	0	6	9	5	8	6	3	2	3	33	8	0	43	2	7	13	13	9	2	2	2	19	6	43	11	5	7	15	3	37	13	5	10	1504	1	5	13	75	1	9	14	
U.S. NEWS	0	0	6	15	0	0	31	0	0	1	20	3	0	1	1	8	4	2	2	2	1	1	1	0	74	1	5	3	13	0	0	0	4	3	16	28	0	10	11	1	7	0	
WEDDINGS	1	0	1	6	0	2	2	0	9	0	20	3	2	4	0	0	3	4	0	0	0	4	1	3	7	7	1	0	4	2	11	1	0	1	15	0	573	3	35	6	0	0	
WEIRD NEWS	1	1	2	7	2	28	43	0	2	0	45	5	0	21	14	19	5	3	2	0	6	2	10	5	35	3	1	12	27	6	8	2	5	7	32	5	6	135	36	7	3	2	
WELLNESS	9	3	7	62	5	17	6	4	19	9	57	11	25	65	11	13	207	26	33	1	5	9	108	18	72	15	21	22	28	8	31	12	12	5	71	0	11	7	2502	26	8	8	

	ARTS	ARTS & CULTURE	BLACK VOICES	BUSINESS	COLLEGE	COMEDY	CRIME	CULTURE & ARTS	DIVORCE	EDUCATION	ENTERTAINMENT	ENVIRONMENT	FIFTY	FOOD & DRINK	GOOD NEWS	GREEN	HEALTHY LIVING	HOME & LIVING	IMPACT	LATINO VOICES	MEDIA	MONEY	PARENTING	PARENTS	POLITICS	QUEER VOICES	RELIGION	SCIENCE	SPORTS	STYLE	STYLE & BEAUTY	TASTE	TECH	THE WORLDPOST	TRAVEL	U.S. NEWS	WEDDINGS	WEIRD NEWS	WELLNESS	WOMEN	WORLD NEWS	WORLDPOST
WOMEN	3	3	12	15	7	5	6	1	8	1	77	0	4	9	0	3	21	3	7	1	5	1	30	14	107	10	5	2	11	4	20	2	1	1	13	3	7	2	73	213	3	1
WORLD NEWS	2	1	4	12	0	0	15	1	0	1	11	3	0	0	2	9	1	1	8	1	4	1	6	1	92	7	18	4	6	0	4	1	4	116	15	5	3	4	13	6	237	41
WORLDPOST	2	1	4	7	0	2	8	1	2	3	11	2	2	2	0	5	10	0	17	0	4	1	3	2	84	3	1	1	5	1	3	0	3	85	26	0	1	5	24	6	48	131

Predicted label

```
In [65]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import numpy as np

top10 = df['category'].value_counts().head(10).index.tolist()

mask = np.isin(y_test, top10)

y_true = y_test[mask]
y_pred = y_pred_svm[mask]

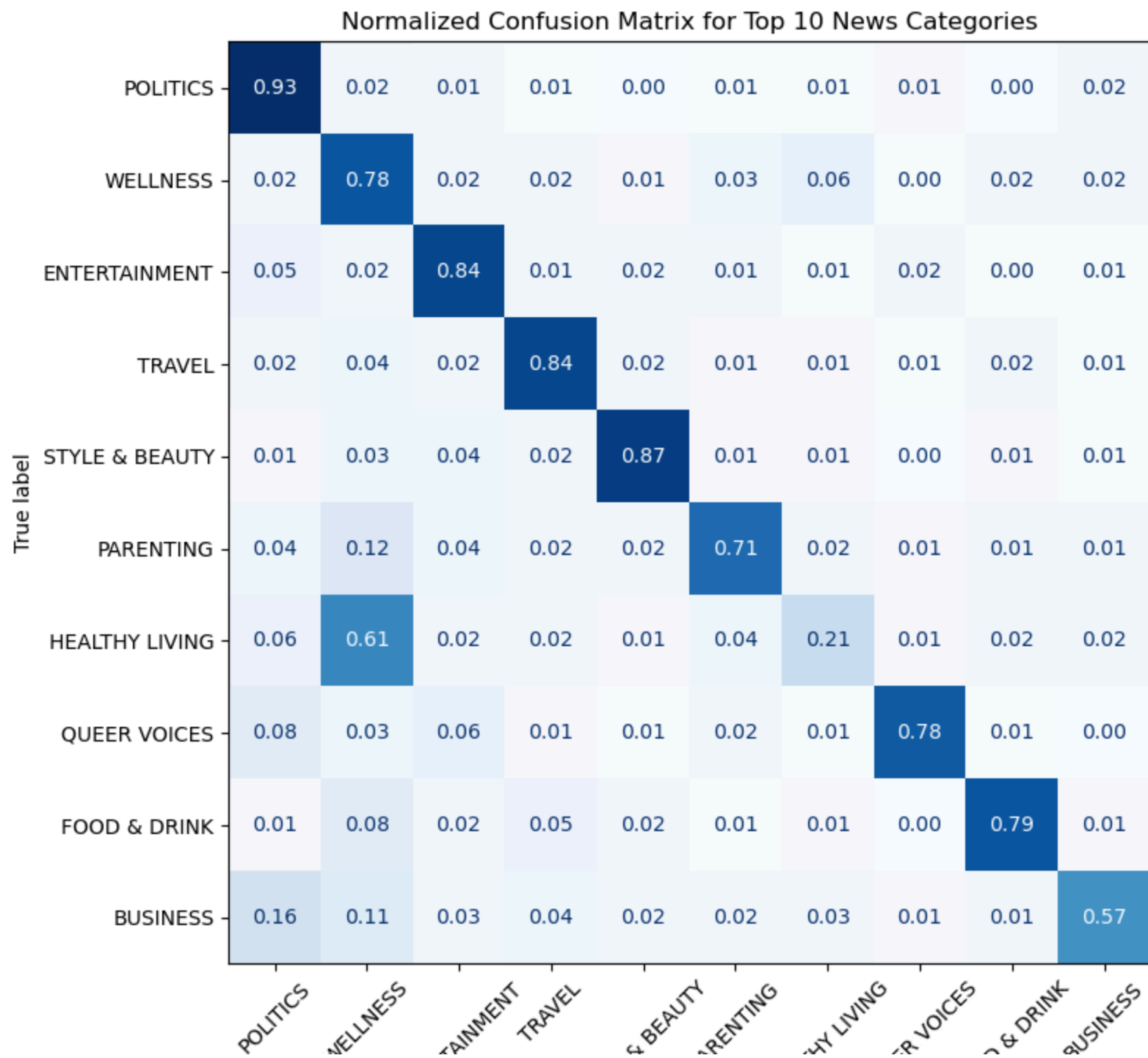
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=top10,
    normalize='true'
)

fig, ax = plt.subplots(figsize=(10,8))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=top10
)

disp.plot(
    cmap='Blues',
    ax=ax,
    colorbar=False,
    xticks_rotation=45,
    values_format='.2f'
)
```

```
plt.title("Normalized Confusion Matrix for Top 10 News Categories")  
plt.tight_layout()  
plt.show()
```



ENTERI.
STYLE
Ph
HEALI,
QUEE.
FOOL
L

Predicted label

In []: